Intelligent Parent Dashboard — project handover

Prepared 10 November 2026. Editable source: `docs/HANDOVER.md`; shareable copies: `docs/HANDOVER.docx` and `docs/HANDOVER.pdf`. Regenerate copies from `backend/` using `python -m scripts.export_handover` (requires LibreOffice). This is the current working prototype handover, including Phase 2A assignments, Phase 2B competency tracking, Phase 2C private messaging and Phase 2D school-wide reporting. It distinguishes delivered functionality from proposal requirements and future validation. The repository contains uncommitted work from several build sessions; this document does not imply a release commit or deployment.

Project purpose and present status

The application demonstrates an administrator setting up authorized school relationships, a teacher recording learner information, and a linked parent receiving monitoring updates and advisory guidance. The core demonstration is implemented. Full proposal compliance and production readiness remain later milestones.

The implementation deliberately retains **FastAPI/Python, SQLAlchemy, MySQL, vanilla HTML/CSS/JavaScript, JWT and the existing UCI-trained CatBoost model**. The proposal described Node.js/Express and React; the written proposal/final report must acknowledge this technology change. No Kenyan learner dataset was available for local model validation. Fictional demonstration records establish software behaviour, not Kenyan predictive accuracy.

Delivered functionality

Area	What works now
Accounts and authorization	Admin/teacher/parent login; administrator account creation/edit/deactivation/restoration; role and relationship checks; session invalidation after sensitive account changes
School relationships	Learner enrollment/profile edit/archive/restore; teacher class/subject assignment and parent/learner link creation, reading, editing and removal
Teacher records	Assessment creation/history/correction/deletion; daily attendance register loading/saving/correction/removal; behaviour ratings and observations with correction/deletion
Analytics reliability	Shared calculation pipeline; automatic refresh after relevant writes; assessment cohort percentile effects; durable freshness; saved records survive model failure; manual calculation retry
Parent monitoring	Linked-learner selection; academic year/term; recorded assessment history and attendance totals/trends; visible-page polling every 30 seconds and manual refresh;

	loading, empty and unavailable states
Recommendations	Academic, attendance and behaviour expert rules; resource-backed learning paths; benchmark academic-risk indicator with explicit limitations
Adaptive notifications	In-app linked-recipient alerts; preference categories, detailed/concise format, priority handling, duplicate suppression and read-state management
Phase 2A — assignments	Teacher class-work CRUD via edit/withdraw/restore; completion registers; submission dates and late reasons; automatic missing status; excused work; parent completion rates/guidance; enrollment/transfer handling
Phase 2B — competencies	Teacher-defined skill catalog; four rubric levels; dated evidence history; correct/delete observations; archive/restore skills; parent latest levels, change labels and practice guidance; linked-parent support follow-ups
Phase 2C — private messaging	Private learner/parent/teacher conversations; immutable text, paginated history, read cursors/unread counts; safe retries; generic in-app alerts; read-only retained history after school links end; no administrator content access
Phase 2D — school reports	Administrator period/grade/stream reports; school/grade/class/subject summaries; attendance trends; assignment and competency coverage; valid-current support totals; previous-period comparisons; learner follow-up register and authenticated CSV exports
Correction accountability	Protected administrator change history containing before/after snapshots; removals of teacher evidence retain an audit entry
Reproducibility	Linux environment/dependency instructions, explicit repeatable fictional seed, isolated automated tests and browser rehearsal scripts

Assignments and competency guidance are independent of the predictive model. Competency evidence is entered by a teacher, not inferred from assessment scores. The generic teacher-defined catalog is not a verified official curriculum catalog.

Architecture and code landmarks

The buildless frontend is served by FastAPI at `/portal`. The API is mounted under `/api`; interactive OpenAPI documentation is at `/docs`.

Location	Responsibility
`backend/app/main.py`	App setup, router registration, additive table creation, static frontend mount
`backend/app/core/`	Database configuration, JWT/dependencies and term date

	windows
`backend/app/models/` and `schemas/`	SQLAlchemy persistence and request/response validation
`backend/app/routers/admin.py`, `admin_crud.py`	User/learner/relationship management and protected correction history
`backend/app/routers/teacher.py`, `teacher_crud.py`	Teacher records and corrections
`backend/app/services/analytics_service.py`	Invalidation, freshness, shared refresh and durable source-record handling
`backend/app/services/percentile_service.py`	Active grade/stream cohort calculations
`backend/app/services/expert_system.py`, `learning_path_service.py`	Advisory rules and learning paths
`backend/app/services/notification_service.py`	Recipient preferences, priority and duplicate suppression
`backend/app/services/lifecycle_service.py`	Active/archive state, authorization helpers and audit snapshots
`backend/app/routers/assignments.py`, `services/assignment_service.py`	Class-work completion, roster handling, separate guidance and alerts
`backend/app/routers/competencies.py`, `services/competency_service.py`	Skill/evidence CRUD, rubric summaries, separate guidance and alerts
`backend/app/routers/messaging.py`, `services/messaging_service.py`	Private messaging, participant privacy, current write eligibility, retry deduplication and post-commit alerts
`backend/app/routers/reports.py`, `services/reporting_service.py`	Read-only administrator reporting, explicit denominators, previous-period comparison, filtered learner pages and spreadsheet-safe exports
`frontend/JS/`	Dashboard controllers; directory is uppercase on Linux
`frontend/JS/record-editor.js`	Shared teacher/admin correction dialog
`backend/tests/`	Isolated API/service/integration checks
`backend/scripts/`	Explicit fictional seeds and browser rehearsals

`ml_training/models/`, `ml_training/reports/`	Existing model artifacts, metadata and benchmark evaluation

Teacher source writes commit before synchronous calculations/notification attempts. Assessment changes refresh the affected cohort because changing one score can affect classmates' percentiles. Attendance/behaviour changes refresh the relevant learner/rules. The API withholds dependent stale prediction outputs and exposes last calculation time; a failed prediction must never undo the teacher's record.

New tables use the existing `Base.metadata.create_all` mechanism: analytics freshness, lifecycle/audit, resources/learning paths, notifications, class work/completions, competencies/evidence, and private conversations/messages. This creates missing tables but **is not a versioned migration system** and will not alter existing columns. Add/import models before schema creation. A production migration strategy is still needed.

Running the application

Use Python 3.12 (tested). From the project directory:

```
python3 -m venv backend/.venv-linux
source backend/.venv-linux/bin/activate
python -m pip install -r backend/requirements-demo.lock
cd backend
```

The existing `backend/.venv` contains Windows files; use a fresh environment on Linux. `requirements-demo.lock` pins the rehearsed runtime/test/browser dependencies; `requirements-dev.txt` allows development version ranges; `requirements.txt` is runtime-only.

Create a MySQL database and a database user allowed to create and use its tables. Configure `backend/.env` from `.env.example`, or export `DATABASE_URL` and a private `JWT_SECRET_KEY`. Shell environment values take precedence. Preserve any existing credentials; do not point seeds/tests at a real school database.

```
# Explicit fictional fixtures; preserves matching source data and existing passwords.
python -m scripts.seed_demo --term 'Term 3' --year 2026 --with-assignments --with-competencies --with-messages
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Open `http://127.0.0.1:8000/portal/login.html`. No npm build is required. `/portal` uses the same API origin, including custom ports. Health check: `/api/health`.

The current local rehearsal uses port **18080** and an isolated MySQL instance under `/tmp/ipd-mysql-demo` (socket `/tmp/ipd-mysql-demo/mysql.sock`, database `ipd_demo`). The original configured MySQL connection was unavailable; its credentials were not overwritten. This temporary instance is not a durable deployment. Back up/export any work you intend to retain before removing temporary data or rebuilding the environment. Configure your own persistent MySQL database for an ongoing installation.

Fictional demonstration accounts

Role	Username	Initial password

Administrator	`demo.admin`	`DemoSchool2026!`
Teacher	`demo.teacher`	`DemoSchool2026!`
Parent	`demo.parent`	`DemoSchool2026!`

Existing passwords are never reset by the seed. Demo teacher teaches Mathematics/English in Grade 7, Demo East. Demo parent is linked to Amina and David Demo. David demonstrates missing academic assessment data while assignment/competency monitoring still works. Esther Demo in Demo West demonstrates class access restrictions. All these names and records are fictional.

Use Term 3 / 2026 for the rehearsed fixtures. Seeds are additive, explicitly invoked, and collision-checked; they are not a database reset. Repeating the basic seed recalculates demo analytics. Assignment/competency seed helpers preserve existing teacher corrections; their fixture lifecycle is documented separately.

Suggested demonstration sequence

1. As admin, inspect the teacher assignment and the parent's links. Show that the parent has only linked learners.

2. As teacher, select Demo East / Mathematics / Term 3 2026. Save/correct an assessment; show refreshed average, percentile, support indicator and recommendations. Show a model failure/pending state without losing records if needed.

3. Record attendance/behaviour; show appropriate rule-based guidance.

4. Create class work, mark Amina submitted late with an actual date and reason, then review missing work and completion rates.

5. Open Competency tracking. Record explicit evidence against a skill, then add a later observation showing a changed level. Demonstrate Not assessed, correction and archive/restore.

6. Open Messages in both dashboards. Send in both directions, show read status and a generic alert, then explain retained read-only history after a school link ends. Administrators cannot inspect message content.

7. Keep the parent's browser visible. Confirm updates arrive within one 30-second cycle; demonstrate manual refresh and term selection. Open assignment and competency guidance and notifications.

8. As administrator, open School Reports. Filter by term/class, inspect recorded-data coverage and current analytics, compare the previous term, use the learner follow-up selector and download an all-matching CSV. Explain current-enrollment and unassessed-data limitations.

9. Review the same flow at a mobile width. Explain that the model is an unvalidated local-context benchmark prototype and evidence is teacher-recorded.

Tests and evidence

From `backend/` with the environment active:

```
python -m pytest tests -q
python -m playwright install chromium
python -m scripts.browser_demo --base-url http://127.0.0.1:8000
python -m scripts.browser_crud_audit --help
python -m scripts.browser_lifecycle --help
python -m scripts.browser_assignments --base-url http://127.0.0.1:8000
python -m scripts.browser_competencies --base-url http://127.0.0.1:8000
python -m scripts.browser_messaging --base-url http://127.0.0.1:8000
python -m scripts.browser_reports --base-url http://127.0.0.1:8000
```

Automated tests use disposable SQLite databases; CRUD fixture tests can optionally target a disposable MySQL database with `CRUD_TEST_DATABASE_URL`. The fixture rejects non-SQLite database names that do not end in `_crud_test`, and drops/recreates tables in the explicitly selected disposable database. Never reuse the application's database for this. Browser scripts add fictional data and may edit/archive demo records; use a separate rehearsal database.

The Phase 2D full regression run passed **161 tests**, with **1 skipped** existing MySQL-only concurrency test. All **17 reporting-specific checks** passed separately on isolated MySQL, followed by focused final-query/year/blank-stream checks. Chromium verified administrator filters, comparisons, all-matching CSV, empty/error recovery and desktop/mobile layout. Report reads were verified to preserve database record counts and prediction snapshots. Python compilation and modified JavaScript syntax checks passed. See validation results (validation.md) for exact checks and scope.

Earlier checkpoints: 96 tests after CRUD completion, 110 after assignments, 127 after competencies (126 full-run plus one focused follow-up), and 144 passed/one MySQL-only skip after messaging. Seventeen competency cases and all 18 messaging cases—including concurrent send/alert handling—also passed on MySQL. These are functional checks, not Kenyan model validation, stakeholder usability acceptance, load testing or production security certification.

Model inputs, evaluation and boundaries

The existing benchmark model uses exactly three inputs: term assessment average (%), grade/stream cohort percentile, and absences scaled to the benchmark full-year horizon. Assessment writes change inference inputs, **not trained model weights**. Behaviour informs expert rules. Assignments, competencies and messages do not enter CatBoost. No retraining was performed during these implementation phases.

The UCI target is final grade `G3 < 10/20`. Reported held-out benchmark metrics are accuracy 89.95%, precision 79.07%, recall 73.91%, F1 76.40%, ROC AUC 0.9599. These are existing report results, not measurements on Kenyan students or a new evaluation in this phase. See evaluation methodology (../ml_training/reports/evaluation.md), including calibration limitations. Displayed risk is an advisory academic support indicator, not validated Kenyan failure probability or behavioural prediction. Term-to-year absence scaling and later-term use are prototype assumptions.

Remaining work and recommended order

School-wide reporting is delivered. The recommended next functional build is intervention plans and follow-through: responsibility, review dates, recorded actions and outcomes.

Priority	Work left	Completion should mean

Next functional phase	Intervention outcomes and review cycles	Assigned support actions, responsible person, review date, completion/evidence, and measured outcome tracking
Curriculum refinement	Verified structured curriculum catalog	Teacher/stakeholder-reviewed competency definitions, curriculum references/versioning, grade progression and historical enrollment policy; keep custom rubrics explicit
Project submission	Proposal/report alignment	Explain technology departure; reconcile delivered/deferred requirements; update ERD/screenshots/test evidence; explain model limitations without Kenyan-accuracy claims
Local validation	Appropriate Kenyan pilot data and evaluation	Lawful school participation/data handling, representative evaluation design, external/local validation and subgroup/calibration checks before accuracy claims; no synthetic-data substitute
Operational reliability	Migration/backup/ background-work strategy	Versioned migrations, restore rehearsal, durable delivery retries, scheduled reminders if required, multi-user write conflict handling and asynchronous large-cohort recalculation
Production readiness	Deployment and security review	Managed secrets, HTTPS, restrictive CORS, operational logging/monitoring, password recovery/account policy, privacy/retention review and measured performance/load checks
User acceptance	Real teacher/parent/admin review	Agreed success tasks, accessibility/mobile usability feedback, corrected terminology and documented acceptance

Current limits include simplified fixed calendar term windows rather than annually configured school calendars; synchronous small-cohort recalculation; in-app notifications rather than external SMS/email; no scheduled assignment/competency reminder worker; no complete historical enrollment register or versioned competency catalog; and no verified official competency coverage. Private messages are text-only and immutable, with in-memory unsent drafts and no attachment storage or durable alert retry worker. Historical participant access is deliberate; removing school links prevents replies rather than erasing past conversation access. Report comparisons use current active enrollment and recorded denominators, not official school-day or curriculum coverage; large-school reporting load tests and indexes remain pending. See school reports (school-reports.md), messaging (messaging.md), assignment tracking (assignment-tracking.md) and competency tracking (competency-tracking.md) for precise transfer/archive/denominator semantics.

Starting the next work session

Read this handover, README and validation results (validation.md). Check `git status` and preserve existing uncommitted work and model artifacts. Start a disposable test database and run the existing suite before changing behaviour. Implement the next agreed phase using current role/link/lifecycle helpers, retain endpoint compatibility, and add focused authorization/integration tests. Rehearse teacher-to-parent changes on MySQL

and desktop/mobile before updating the feature status and this handover. Do not retrain or claim full proposal completion merely because the demonstration works.